

APKMer Assembler: An Interactive Pipeline for k -mer Optimised Genome Assembly with De Bruijn Graphs and Graph Neural Networks

Poulomi Sarkar & Aditya Malhotra

Graph Theory and Network Science - Term Paper | Spring 2026 | Submitted: April 30, 2026

Abstract

*This paper presents APKMer Assembler, an end-to-end interactive pipeline that integrates read simulation, k -mer parameter optimisation, and machine-learning-guided genome assembly into a single reproducible workflow, developed to investigate how variable k -mer length shapes de Bruijn graph structure and assembly quality. De Bruijn graphs underpin virtually every modern short-read genome assembler, reducing sequence reconstruction to a graph traversal problem over observed k -mers. The choice of k is the most consequential free parameter in this framework, yet most classical assemblers require it to be specified manually. This paper provides a rigorous treatment of de Bruijn graph theory alongside a detailed review of KmerGenie (Chikhi and Medvedev, 2014), which estimates the optimal k by maximising the count of distinct error-free genomic k -mers, and COGRAM (Coggins and Ramasamy, 2025c), which trains a lightweight Graph Convolutional Network to score de Bruijn graph edges and decode genomic paths via a hybrid greedy-plus-beam search. The pipeline is demonstrated on the genome of *Nasuia deltocephalincola*, confirming that small k produces an over-compressed graph, intermediate k maximises branching density, and large k induces fragmentation—and that the KmerGenie-selected k falls consistently within the resolving intermediate range.*

1. Introduction

Genome assembly, the reconstruction of a DNA sequence from short sequencing reads, is a central problem in computational biology. Modern assemblers predominantly formulate this problem using *de Bruijn graphs*, where reads are decomposed into k -mers and assembly reduces to a graph traversal problem. Within this framework, the choice of the k -mer length k is the most critical free parameter: it directly determines graph connectivity, repeat resolution, and susceptibility to sequencing errors.

Selecting an appropriate k involves a fundamental trade-off. Small k values produce highly connected graphs with extensive ambiguity due to repeat collapse, while large k values reduce ambiguity but fragment the graph due to coverage gaps and error sensitivity. Despite its importance, there is no closed-form solution for the optimal k , and most assemblers require it to be specified manually, often through heuristic trial-and-error.

Several approaches attempt to address this limitation. *KmerGenie* (Chikhi and Medvedev, 2014)

provides a data-driven estimate of the optimal k by analysing k -mer abundance histograms, while recent methods such as *COGRAM* (Coggins and Ramasamy, 2025b) introduce machine learning techniques to guide graph traversal. However, these tools exist as separate components: parameter selection, graph construction, and assembly are typically performed in isolation, requiring significant manual coordination.

In this paper, we present **APKMer Assembler**¹, an end-to-end interactive pipeline that integrates read simulation, k -mer optimisation, and graph-based genome assembly into a single reproducible workflow. The pipeline combines KmerGenie for parameter selection with COGRAM for graph-based assembly, providing a unified framework in which each stage feeds directly into the next. This integration enables systematic experimentation and transparent evaluation of how k influences both graph structure and assembly quality.

To investigate these effects, we construct de Bruijn graphs for k values ranging from 3 to 30 using the genome of *Nasuia deltocephalincola*, and analyse the resulting structural behaviour. Our observations reveal

¹AP stands for Aditya and Poulomi

a consistent pattern: small k leads to over-compression, intermediate k maximises branching ambiguity, and large k induces fragmentation. The k values selected by KmerGenie fall within the intermediate regime where ambiguity begins to resolve, validating its underlying heuristic.

The remainder of the paper is organised as follows. We first introduce the biological and graph-theoretic foundations of genome assembly, followed by a discussion of the k -selection problem. We then present our data exploration, describe the KmerGenie and COGRAM methodologies, and detail the APKMer Assembler pipeline. Finally, we report experimental results and discuss limitations and future directions.

2. A Biology Primer

2.1. What is DNA?

DNA (deoxyribonucleic acid) is a long, thread-like molecule that carries the instructions needed for an organism to grow, function, and reproduce. It is composed of two strands twisted into a double helix, where each strand is a sequence of four chemical units (bases) denoted A, C, G, and T. The specific order of these bases encodes biological information, much like letters form words and sentences. A genome is the complete collection of an organism's DNA, containing all of its genetic instructions.

2.2. What is genome sequencing?

Genome sequencing is the process of determining the exact order of bases in a genome. In practice, current technologies cannot read very long stretches of DNA in one piece, so the DNA is first broken into many small fragments, in a lab. Each fragment is then "read" by sequencing machines, producing short sequences called "reads". This is done to understand genetic variation, study diseases, identify organisms, and explore evolutionary relationships. However, sequencing introduces several challenges: reads are short and may contain errors, and many regions of DNA are repetitive, making it difficult to determine where certain fragments belong.

2.3. What is genome assembly?

Genome assembly is the computational step that follows sequencing, where these short reads are pieced back together to reconstruct the original genome. This is typically done by finding overlaps between reads and merging them into longer sequences called contigs, and then further organizing them into larger structures. The process is complicated by sequencing errors, uneven coverage (some regions are read more times than

others), and repeats that can cause ambiguities in reconstruction. Genome assembly can be formalized as reconstructing a target DNA sequence from a large collection of short, overlapping reads. Modern sequencing technologies generate millions to billions of reads, typically 50 - 300 bases long. The task is therefore analogous to reconstructing a long string from many noisy, overlapping substrings, under constraints imposed by coverage gaps, sequencing errors, and genomic repeats.

2.4. Where does graph theory come in?

A central idea in addressing these challenges is to represent sequencing data using graph-based models, where relationships between fragments can be analyzed systematically.

Classical genome assembly methods followed the *overlap-layout-consensus* (OLC) paradigm, where reads are nodes and overlaps define edges. Reconstructing the genome then amounts to finding a Hamiltonian path, which is computationally expensive and scales poorly for large datasets. The key shift in modern assembly is to instead break reads into k -mers and construct a de Bruijn graph, where nodes are $(k-1)$ -mers and edges correspond to observed k -mers. Assembly is then formulated as finding an Eulerian path through the graph, which is computationally tractable. This change in formulation is what makes large-scale genome assembly feasible.

3. What are de Bruijn Graphs?

A *de Bruijn graph* of order n over an alphabet Σ of size k , denoted $B(k, n)$, is a directed graph whose nodes are all possible strings of length $n-1$ over Σ , and whose edges represent all possible strings of length n . Specifically, there is a directed edge from node u to node v if and only if the last $n-1$ characters of u equal the first $n-1$ characters of v ; the edge is labelled with the string formed by appending the last character of v to u (Compeau et al., 2011).

The graph $B(k, n)$ has k^{n-1} nodes and k^n edges. A fundamental property of this construction is that every node has in-degree equal to out-degree equal to k . A directed graph in which this balance holds at every node is called *Eulerian*, and it admits a closed walk—an *Eulerian cycle*—that traverses every edge exactly once. In $B(k, n)$, such an Eulerian cycle spells out a cyclic sequence that contains every possible n -mer over Σ as a consecutive substring exactly once. This is the *de Bruijn sequence* of order n over Σ . The construction will become clear through an example.

Consider the binary alphabet $\{0, 1\}$, i.e. $k = 2$ and $n = 3$. The four possible 2-mers - 00, 01, 10, 11 - become

the four nodes of $B(2,3)$. The eight possible 3-mers become the eight directed edges. Because each node has in-degree and out-degree equal to 2, an Eulerian cycle exists. One such cycle visits the edges in the order $000 \rightarrow 001 \rightarrow 011 \rightarrow 111 \rightarrow 110 \rightarrow 101 \rightarrow 010 \rightarrow 100$, spelling the cyclic binary sequence **00011101**. Reading the first character of each edge label in order yields this sequence, which contains every 3-bit string exactly once.

Figure 1 illustrates this $B(2,3)$ graph.

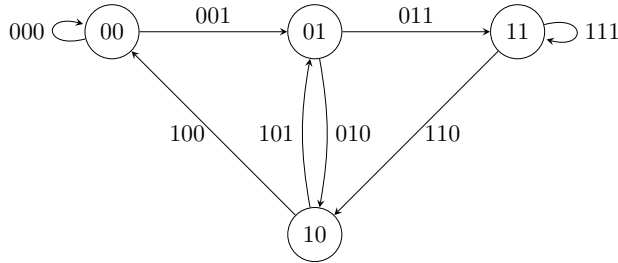


Figure 1. The de Bruijn graph $B(2,3)$ over the binary alphabet. Nodes represent 2-mers; directed edges represent 3-mers. Following an Eulerian cycle produces the sequence 00011101, which contains every 3-bit string exactly once.

De Bruijn graphs satisfy some nice properties which make them ideal candidates for tasks such as genome assembly:

- Euler’s theorem states that a connected directed graph possesses an Eulerian cycle if and only if every node has equal in-degree and out-degree. Since $B(k,n)$ satisfies this condition by construction, an Eulerian cycle always exists (Compeau et al., 2011).
- Finding a *Hamiltonian* cycle is an NP-complete problem. Finding an *Eulerian* cycle is solvable in linear time via Hierholzer’s (or Euler’s) algorithm. This asymmetry in computational complexity is precisely what makes the de Bruijn graph formulation powerful for genome assembly

3.1. de Bruijn Graphs for Genome Assembly: An Example

To see how de Bruijn graphs are used in practice, consider a set of sequencing reads from a short DNA fragment. Each read is broken into overlapping k -mers (substrings of length k). From these k -mers, we construct a de Bruijn graph where nodes are $(k-1)$ -mers and directed edges represent observed k -mers.

Figure 2 shows such a graph constructed from a collection of reads. Each edge is labelled by a k -mer (e.g., AAT, ATG, TGG), and connects its prefix to its suffix. For instance, the k -mer AAT connects node AA to node

AT. The numbers on the edges indicate multiplicity, i.e., how many times a given k -mer appears in the read set (its coverage).

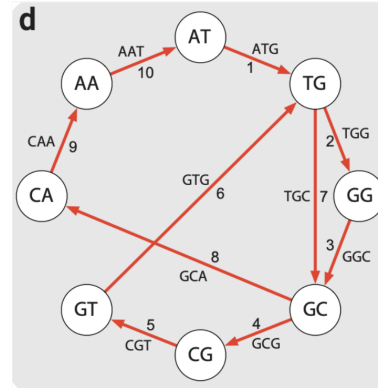


Figure 2. A de Bruijn graph constructed from sequencing reads. Nodes represent $(k-1)$ -mers and edges represent observed k -mers, with weights indicating multiplicity.

Genome assembly in this framework corresponds to finding a path through the graph that uses edges in a manner consistent with their multiplicities. In the ideal case, this reduces to finding an Eulerian path, which traverses every edge exactly once. Reading off the sequence along such a path reconstructs the original DNA sequence from which the reads were derived.

This example illustrates how local overlaps between short reads are encoded globally in the graph structure, allowing the reconstruction problem to be solved efficiently.

3.1.1. Assumptions and Practical Limitations

The effectiveness of de Bruijn graph assembly relies on several idealised assumptions:

- **Complete k -mer coverage:** Every k -mer in the genome is assumed to appear in the read set. In reality, coverage is uneven and gaps may exist.
- **Error-free reads:** Sequencing errors introduce incorrect k -mers, which create spurious branches (e.g., bubbles and tips) in the graph.
- **Uniqueness of k -mers:** Repeats longer than k collapse onto the same nodes, leading to ambiguities that cannot be resolved without additional information.
- **Single contiguous sequence:** The ideal model assumes a single sequence, whereas real genomes may consist of multiple chromosomes, requiring separate paths.

These deviations from the ideal model necessitate additional algorithmic techniques in practical assemblers, but the core graph-based formulation remains central.

4. The Problem of Optimal k

4.1. Competing Effects of k

The k -mer length k is the most consequential free parameter of any de Bruijn graph assembler. Its selection involves a fundamental trade-off between two opposing effects.

Small k : Short k -mers are more likely to be observed in sequencing data (higher sensitivity), but are also more likely to occur in multiple genomic locations (lower specificity). A very small k produces a densely connected graph in which the same node corresponds to many genomic positions, causing severe branching and ambiguity. Repeats longer than k bases completely collapse into the same path, making them irresolvable.

Large k : Long k -mers are more genomically unique, resolving more repeats and reducing branching. However, a longer k -mer is more likely to contain a sequencing error, and read pairs that overlap by fewer than k bases no longer share a node in the graph. This creates coverage gaps that break contigs, fragmenting the assembly (Chikhi and Medvedev, 2014).

The trade-off is summarised by the observation: *small k helps you see everything but makes it hard to distinguish things; large k helps you distinguish things but makes you miss things.*

4.2. Absence of a Closed-Form Optimum

Despite the intuitive clarity of this trade-off, there is currently no closed-form mathematical formula for the optimal k given a genome and a read dataset. Lander–Waterman statistics can provide rough bounds based on genome size and coverage depth, but they do not account for genome heterozygosity, repetitiveness, or base-call error rates (Chikhi and Medvedev, 2014). In practice, k is often chosen based on prior experience with similar datasets or by trying several values and comparing the resulting assemblies—an approach that is both time-consuming and genome-specific.

Chikhi and Medvedev (2014) propose a principled solution: the best k is the one that yields the maximum number of *distinct, error-free (genomic) k -mers* in the sequencing data. This heuristic is formalised in the KMERGENIE tool and is discussed in Section 6.

5. Data Exploration: de Bruijn Graphs on a Real Genome

To build intuition for how de Bruijn graphs behave in practice, we conducted a focused data exploration using the genome of *Nasua deltocephalincola*. Reported in 2013 as having one of the smallest known bacterial genomes, it consists of 112,590 base pairs, making it a convenient testbed: small enough for fine-grained analysis across many k values, yet still exhibiting features such as repeats, branching, and fragmentation. The genome sequence (FASTA format) was obtained from the NCBI database.

5.1. Graph Construction Across k

For each integer k from 3 to 30, a de Bruijn graph $G_k = (V_k, E_k)$ was constructed from the genome sequence. The procedure is straightforward:

- Step 1.* Read the genome sequence in $O(n)$ time, where $n = 112,590$.
- Step 2.* Generate all overlapping k -mers using a sliding window.
- Step 3.* Construct a directed graph where nodes are unique $(k-1)$ -mers and edges correspond to observed k -mers (prefix $\rightarrow$ suffix). Multiple edges are allowed to capture repeated k -mers (Compeau et al., 2011).

The implementation was carried out in Python using NetworkX (DiGraph). For a fixed k , both k -mer generation and graph construction scale linearly with genome length.

5.2. Exploratory Metrics

To probe how the graph structure evolves with k , a small set of structural quantities was computed for each G_k :

- **Node and edge counts** ($|V_k|, |E_k|$): number of distinct $(k-1)$ -mers and observed k -mers.
- **Branching nodes:** nodes with out-degree > 1 , indicating multiple possible continuations.
- **Dead ends:** nodes with out-degree 0.
- **Strongly connected components (SCCs):** computed using Tarjan’s algorithm in $O(|V| + |E|)$ time.

From these, simple derived quantities such as branching density and fragmentation (number of SCCs relative to $|V_k|$) were also examined.

5.3. Empirical Observations

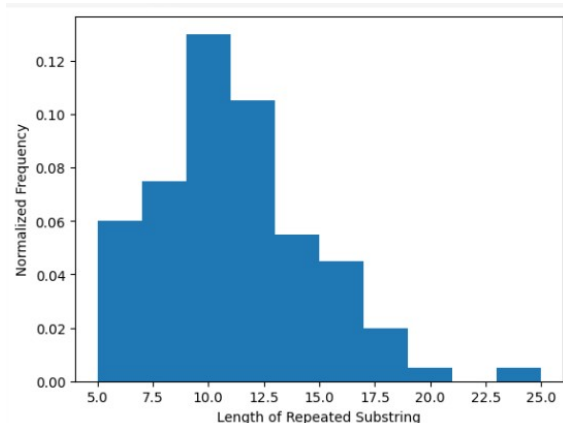


Figure 3. Distribution of repeated substring lengths in the *N. deltocephalincola* genome.

The distribution of repeated substring lengths provides a direct view of the repeat structure of the genome. Most repeats are short (peaking around length ~ 10), with a long but sparse tail extending to larger lengths. This is consistent with the expectation that short repeats are common, while long exact repeats are rare. From a de Bruijn graph perspective, this suggests that k must exceed the typical repeat length (around 10–15) to resolve most ambiguities.

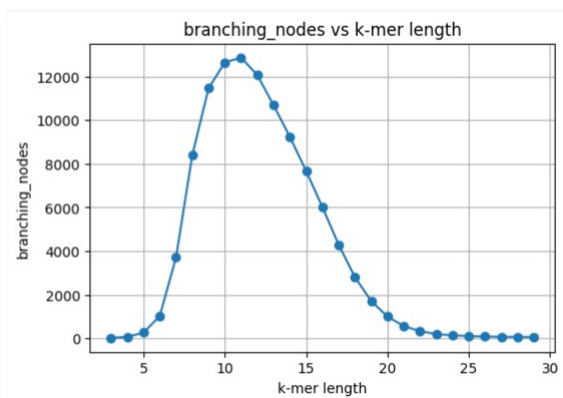


Figure 4. Number of branching nodes as a function of k -mer length.

The number of branching nodes as a function of k exhibits a clear unimodal behaviour. For small k , the graph is highly compressed and branching is minimal. As k increases, branching rises sharply and peaks around $k \approx 10$ –11, reflecting the regime where repeats are long enough to create ambiguity but not long enough to be resolved. Beyond this point, branching decreases steadily as k -mers become increasingly unique, eventually approaching zero for large k where the graph

becomes nearly linear.

These observations align with the structural trends expected from de Bruijn graphs and provide an empirical indication of the trade-off in choosing k .

5.4. Observed Behaviour Across k

Varying k reveals clear structural trends in the resulting graphs:

- **Small k :** the graph is highly compressed, with many sequences merging into the same nodes. This leads to dense connectivity but poor resolution.
- **Intermediate k :** branching becomes prominent, reflecting ambiguity due to repeated subsequences.
- **Larger k :** many k -mers become unique, reducing branching and producing longer linear paths.
- **Very large k :** overlaps become scarce and the graph fragments into many small components.

These trends reflect the idea that there is an optimal k value that is neither too high nor too low. The sweep of k values from 3 to 30 gives us an idea that for this particular genome sequence the best k lies in the intermediate range of 18 – 22. A rigorous test of what the best k requires further analysis.

6. KmerGenie: An algorithm for selecting best k

The empirical trends above highlight a central question: how should one choose an appropriate k ? One principled approach is provided by *KmerGenie* (Chikhi and Medvedev, 2014), which estimates a suitable k directly from sequencing data.

The core idea behind *KmerGenie* is to analyze the abundance histogram of k -mers. For a fixed k , the algorithm counts how many times each distinct k -mer appears and constructs a histogram of these frequencies. This histogram typically exhibits two regimes: low-frequency k -mers arising from sequencing errors, and higher-frequency k -mers corresponding to true genomic sequences. *KmerGenie* fits a statistical model to this distribution to estimate the number of *solid* (error-free) k -mers.

By repeating this process across a range of k values, the algorithm predicts, for each k , the number of genomic k -mers that would contribute to a correct assembly. Intuitively, small k values underrepresent the genome due to excessive merging, while large k values suffer from fragmentation and error sensitivity; the optimal point balances these effects.

7. COGRAM: A Graph-Learning Approach to Genome Assembly

7.1. Overview and Motivation

KmerGenie provides a principled, data-driven estimate of the optimal k -mer length, but it is purely a parameter-selection tool—it does not itself assemble the genome. Once the best k is identified, an assembler must still construct the de Bruijn graph, resolve ambiguous branches, and extract contiguous sequences. Classical assemblers such as Velvet (Zerbino and Birney, 2008), ABySS (Simpson et al., 2009), and SPAdes (Bankevich et al., 2012) accomplish this through heuristic rules: bubble-popping to remove sequencing artefacts, tip-trimming to remove dead-end branches, and paired-end scaffolding to link isolated contigs. These heuristics are highly effective, but their path-selection logic is not learned from data and can misresolve locally complex regions of the graph.

COGRAM (Coggins–Ramasamy Genomic Assembly Method) is a recent Python-native pipeline that takes a different route (Coggins and Ramasamy, 2025b,a). Rather than relying on hand-crafted traversal rules, COGRAM trains a lightweight Graph Convolutional Network (GCN) to score each edge in the de Bruijn graph based on whether it represents a true genomic adjacency or a spurious overlap introduced by repeats or sequencing noise. This scored graph is then decoded using a hybrid greedy-plus-beam search to reconstruct genomic paths.

7.2. The COGRAM Pipeline

The COGRAM framework is organised into six sequential algorithmic stages, illustrated conceptually in Figure 5.

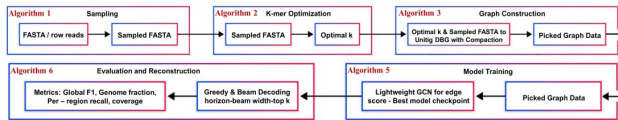


Figure 5. The end-to-end COGRAM pipeline. Arrows indicate data flow; algorithm labels (1–6) mark each computational stage. Figure reproduced from Coggins and Ramasamy (2025b).

7.2.1. Stage 1: Genomic Sampling

Because full bacterial chromosomes can still produce graphs with millions of nodes, COGRAM begins by randomly sampling contiguous windows of a specified length (e.g., 10,000 bp) from the input FASTA file.

7.2.2. Stage 2: k -mer Optimisation

COGRAM includes its own naive k -mer optimiser (Algorithm 2 in Coggins and Ramasamy (2025b)). The algorithm scans a candidate range of k values, counts k -mer abundances across the sampled FASTA, filters

out singleton k -mers (likely sequencing errors), and selects the k whose estimated count of *genomic* k -mers (abundance $\geq$ threshold) is maximal. This criterion mirrors the reasoning behind KmerGenie (Chikhi and Medvedev, 2014) and produces a data-informed k without requiring a Linux-specific installation.

7.2.3. Stage 3: Graph Construction and Unitig Compression

Given the selected k , the sampled FASTA is transformed into two complementary graph objects:

- A **raw k -mer overlap graph** whose nodes are $(k-1)$ -mers and whose edges represent observed k -mer transitions. This graph retains full granularity for model training.
- A **unitig-compressed graph** in which maximal non-branching paths are collapsed into single nodes. Unitig compression reduces the graph by several orders of magnitude while preserving all branching topology, accelerating downstream processing.

Edge labels (true or spurious) are derived from the known order of k -mers in the reference sequence, enabling supervised training. One-hot feature vectors of dimension $4k$ encode each $(k-1)$ -mer node.

7.2.4. Stage 4: GCN Training

The heart of COGRAM is an edge classifier built on two GCN layers followed by a two-layer multi-layer perceptron (MLP). For each directed edge (u, v) , the node embeddings $h_u^{(2)}$ and $h_v^{(2)}$ produced by two rounds of neighbourhood aggregation are concatenated to form an edge feature vector, which is passed through the MLP to produce a sigmoid edge score $\hat{p}_e$. Training minimises binary cross-entropy loss with an Adam optimiser, learning-rate scheduling, and early stopping on a 70/15/15 train-validation-test split.

The GCN propagation rule exploits the symmetric normalised adjacency $\tilde{A} = \tilde{D}^{-1/2} \hat{A} \tilde{D}^{-1/2}$ (where $\hat{A} = A + I$), so each node's embedding after layer l integrates its own features with those of its 1-hop neighbourhood:

$$H^{(l+1)} = \phi(\tilde{A} H^{(l)} W^{(l)}), \quad (1)$$

where ϕ denotes a ReLU activation and $W^{(l)}$ is a trainable weight matrix. Backpropagation through this formulation propagates error signals across graph neighbourhoods, enabling the model to learn local overlap patterns from the training data (Coggins and Ramasamy, 2025b).

7.2.5. Stage 5: Greedy-Plus-Beam Decoding

Assembly is framed as a Hamiltonian reconstruction on the unitig graph. Decoding proceeds in two phases: a *greedy phase* that follows the highest-scoring outgoing edge at each step until a user-defined fraction

of nodes has been visited, followed by a *beam-search phase* at remaining branch points that maintains a set of top- k candidate paths. This hybrid strategy trades computational cost for robustness: enlarging the beam width and top- k parameter improves recall at the cost of additional traversal time.

7.2.6. Stage 6: Evaluation

Assembly quality is reported globally and per sampled region using precision, recall, F1, and genome coverage (fraction of nodes visited at least once). On *E. coli* K-12, COGRAM achieves a global F1 of 0.957 and genome coverage of 0.979 using only short-read data, without paired-end or long-read scaffolding (Coggins and Ramasamy, 2025b).

7.3. Comparison with Classical Assemblers

COGRAM deliberately differs from Eulerian assemblers in two key respects. First, it solves a Hamiltonian rather than Eulerian traversal, posing assembly as a node-visiting problem rather than an edge-covering problem. Second, path selection is guided by learned edge scores rather than coverage thresholds and bubble-popping rules. While classical tools such as SPAdes and ABySS routinely exceed 99% genome fraction on bacterial benchmarks, COGRAM’s 97.9% coverage without paired-end information is notable as a proof of concept that GCN-based edge scoring can learn local overlap patterns from data alone (Coggins and Ramasamy, 2025b).

8. Integrating KmerGenie and COGRAM: The APKMer Assembler Pipeline

8.1. Motivation and Problem Statement

The preceding sections establish two important but separate contributions: KmerGenie provides a principled, read-aware estimate of the optimal k -mer size, and COGRAM provides a graph-learning approach to assembling those reads into contigs. A natural question is whether these two tools can be integrated into a coherent, end-to-end workflow in which parameter selection feeds directly into assembly, and assembly quality can be immediately evaluated.

This question is not merely academic. Many widely-used genome assemblers including Velvet (Zerbino and Birney, 2008), SOAPdenovo 2, ABySS (Simpson et al., 2009), and Minia require the user to specify k explicitly. As Chikhi and Medvedev (2014) note, “these tools ask for the k -mer length rather than trying to estimate it.” Consequently, finding the optimal k -mer size is a prerequisite for obtaining a high-quality assembly, and neglecting this step routinely leads to fragmented or over-

collapsed graphs. Furthermore, estimating the genome size from sequencing reads is independently important: scientists are commonly required to report an approximate genome size when submitting a draft genome, and the k -mer spectrum provides exactly this estimate as a by-product of the optimisation step (Chikhi and Medvedev, 2014).

At the same time, even when the optimal k is known, most assemblers require a Linux environment and non-trivial command-line expertise. COGRAM was explicitly designed to be cross-platform, requiring no external assemblers, but it too is primarily a research prototype without a graphical interface. The gap between a theoretically well-motivated pipeline and one that a practitioner can actually run, inspect, and reproduce in a single session motivates the construction of the **APKMer Assembler**: an interactive, stepwise web application that connects read simulation, k -mer optimisation, graph-and-ML assembly, and evaluation into a guided, transparent workflow.

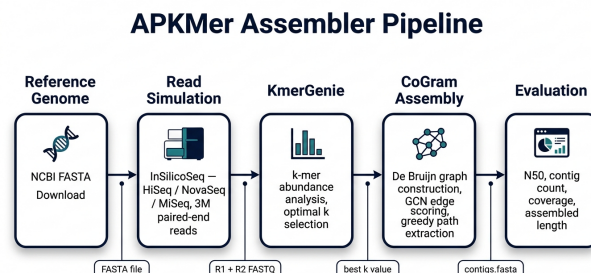


Figure 6. The APKMer Assembler pipeline

The stakes of this integration are concrete. Every choice in the pipeline sequencing model, read count, k -mer length, assembly algorithm affects downstream biological conclusions. An integrated, logged, and restartable pipeline makes these choices explicit and reproducible, which is essential for any result intended for scientific communication.

8.2. System Architecture Overview

APKMer Assembler is built as a Flask-based web application with a Python backend. The backend owns all scientific computation and environment management; the frontend presents one logical stage per page, dispatches jobs to the backend, and streams live log output so the user can monitor progress in real time. Figure 7 illustrates the overall architecture.



Welcome to APKMer Assembler

An interactive genome-assembly workflow with clear step-by-step execution and live system logs.

- Download and prepare reference FASTA input automatically.
- Simulate sequencing reads with configurable models and read counts.
- Estimate optimal k using KmerGenie and review the generated report.
- Run CoGram assembly and inspect final evaluation metrics in one place.

[Proceed to Pipeline](#)

Figure 7. The APKMer Assembler welcome page. Users are guided through the five pipeline stages sequentially. Each stage is locked until the preceding one completes successfully, enforcing correct execution order.

The pipeline is divided into five stages, each with its own page, controls, and log pane:

1. **Reference Preparation** — download and index the reference genome.
2. **Read Simulation** — generate synthetic paired-end reads with InSilicoSeq.
3. **KmerGenie** — estimate the optimal k and review the k -mer report.
4. **COGRAM Assembly** — build the graph, train the GCN, extract contigs, and evaluate.
5. **Results** — inspect assembly metrics and download output artefacts.

8.3. Environment Setup and Reproducibility

8.3.1. Why WSL and Conda

Bioinformatics tooling is primarily distributed and validated for Linux. Running the pipeline on a Windows host machine therefore required the Windows Subsystem for Linux (WSL) as the execution substrate. WSL provides direct access to Linux filesystem semantics, shell scripting, and the Conda/Bioconda package ecosystem, all of which are assumed by tools such as InSilicoSeq and KmerGenie.

8.3.2. Environment Isolation Strategy

A deliberate design decision was made to isolate each pipeline stage into a dedicated Conda environment:

- `sim_env` for read simulation (InSilicoSeq and its Bioconda dependencies).
- `kmergenie_env` for k -mer optimisation (KmerGenie and `setuptools`).
- `COGRAM_env` for graph construction and ML assembly (Python 3.10, PyTorch, PyTorch Geometric).

This separation prevents dependency version conflicts, keeps each environment small and debuggable, and allows targeted upgrades without destabilising the full pipeline.

```
base          * /home/poulomi/miniconda3
cogram_env    /home/poulomi/miniconda3/envs/cogram_env
kmergenie_env /home/poulomi/miniconda3/envs/kmergenie_env
sim_env       /home/poulomi/miniconda3/envs/sim_env
```

Figure 8. Conda environment setup for the three pipeline stages. Each environment is provisioned from Bioconda and Conda-Forge channels. The `-help` flag on each tool confirms successful installation.

8.3.3. Directory Structure

A deterministic project directory layout was established:

- `genomes/` — reference FASTA files.
- `kmergenie_output/` — KmerGenie artefacts (`.dat`, `.histo`, `_report.html`).
- `COGRAM/` — COGRAM source code, model checkpoints, and `contigs.fasta`.
- Root-level FASTQ files produced by the simulation stage.

Canonical, stable paths eliminate the most common class of automation failures (path drift after renaming) and make command arguments in the backend unambiguous.

8.4. Stage 1: Reference Genome Retrieval

8.4.1. Organism Selection and Download

The pipeline begins by obtaining a reference genome sequence. Users select an organism from a dropdown menu in the web interface (Figure 9); the backend then downloads the corresponding compressed FASTA from the NCBI FTP server, decompresses it, and stores it under `genomes/<prefix>.fasta`.

For the experiments in this paper we used *Nasua deltocephalincola*, the same organism studied in Section 5.

Figure 9. Step 1 of the APKMer Assembler: organism selection and reference genome retrieval. The live log pane on the right streams download and decompression progress in real time.

8.4.2. Why This Step Matters

The reference genome serves two purposes in this pipeline. First, it is the source sequence from which synthetic reads are simulated (Section 8.5). Second, it provides ground-truth edge labels for COGRAM’s supervised training: a k -mer transition that appears in the reference is labelled positive; all others are negative. Without a high-quality reference, the supervised training signal degrades.

8.5. Stage 2: Read Simulation

8.5.1. Why Simulate Reads?

Read simulation is not merely a preprocessing convenience—it is the foundational experimental control mechanism for this project. Simulating reads from a known reference genome provides:

- **Deterministic ground truth.** Because the source sequence is known, downstream assembly quality can be measured objectively rather than inferred.
- **Controlled noise.** The sequencing error profile is explicitly parameterised, allowing systematic study of how noise affects graph topology and COGRAM’s edge classifier.
- **Reproducibility.** A fixed random seed and read count yield the same FASTQ files across runs, enabling fair comparisons of different k values or model settings.
- **Safe iteration.** Developing and debugging the pipeline on clean, well-understood data reduces the risk of confounding biological complexity with implementation errors.

Real-world sequencing datasets introduce additional complexity (adapter contamination, uneven coverage, chimeric reads) that would obscure the methodological questions investigated here. Simulation allows those questions to be isolated.

8.5.2. InSilicoSeq and Sequencing Models

Read simulation is performed by InSilicoSeq (ISS) (Gourié et al., 2019), a Conda-installable tool that generates Illumina-like paired-end reads using empirically derived error models. The command template used is:

```
iss generate \
  --genomes <reference.fasta> \
  --model <hiseq|novaseq|miseq> \
  --output <prefix> \
  --n_reads <N> \
  --abundance uniform
```

Three sequencing models are supported: HiSeq (lower error rate, longer reads), NovaSeq (very high throughput), and MiSeq (higher error rate, longer reads).

The `-abundance uniform` flag assumes equal abundance of all input sequences, which is appropriate for single-chromosome bacterial genomes. For the experiments reported here, $N = 3,000,000$ reads were generated, providing sufficient coverage depth for stable k -mer frequency estimates.

8.5.3. Output Artefacts and Naming Convention

ISS produces 2 paired FASTQ files: `<prefix>_R1.fastq` and `<prefix>_R2.fastq`, where the prefix is derived from the organism name :

(e.g., `nasuia_deltoccephalinicola_sim_R1.fastq`).

This organism-based naming avoids hardcoded filenames and supports future multi-organism comparisons without collision.

Figure 10. Step 2 of the APKMer Assembler: read simulation with InSilicoSeq. The user selects the instrument model and read count; the log pane confirms successful generation of both R1 and R2 FASTQ files.

8.5.4. Relationship to Downstream Stages

The simulated reads define the k -mer spectrum that KmerGenie will analyse. Any change in sequencing model or read count can shift the estimated best k , so the exact simulation configuration must be recorded alongside each run. Similarly, the read characteristics directly influence COGRAM’s graph topology: too few reads produce sparse edge support and unreliable training labels; too many reads increase computation time with diminishing returns on assembly quality.

8.6. Stage 3: KmerGenie k -mer Optimisation

8.6.1. Importance of k -mer Selection in This Context

As noted in Section 4, the k -mer length is the most consequential free parameter of any de Bruijn graph assembler, yet most tools require the user to specify it manually. For the COGRAM assembler, this dependence is especially strong because k affects not only graph topology but also the dimensionality of node feature vectors ($4k$), the number of training edges, and the ground-truth labelling derived from reference k -mer transitions. An informed, data-driven k estimate is therefore essential for reproducible COGRAM runs.

8.6.2. Running KmerGenie

The backend optionally merges the R1 and R2 FASTQ files for paired-end analysis before invoking KmerGenie:

```
kmergenie <input.fastq> \
-o kmergenie_output/kmergenie_out
```

KmerGenie scans a range of candidate k values, fits a statistical model to each k -mer abundance histogram to separate error-derived from genomic k -mers, and selects the k that maximises the estimated count of genomic k -mers. It produces a tabular .dat file, per- k histograms, and a visual HTML report.

8.6.3. Parsing and Passing Best k

The backend parses the .dat file to extract the best k and stores it in the global pipeline state, where it is automatically pre-filled into the COGRAM configuration on the next page. This eliminates a manual transcription step that is a common source of error.

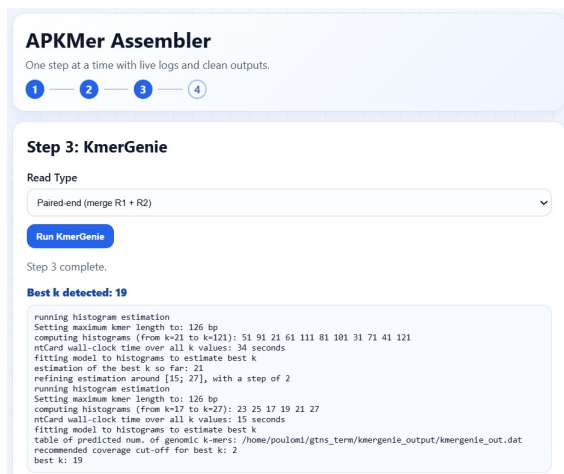


Figure 11. Step 3 of the APKMer Assembler: KmerGenie k -mer optimisation. The embedded HTML report visualises estimated genomic k -mer counts as a function of k ; the detected optimum (here $k = 19$) is surfaced as a labelled card above the report.

8.6.4. Interpreting the KmerGenie Report

The HTML report embedded in Step 3 provides an important sanity check. A well-behaved curve shows a clear global maximum; a flat or noisy curve may indicate insufficient read depth, a highly repetitive genome, or a read count that is too low. Users who observe poor curve quality are encouraged to return to Step 2 and increase the read count or switch to a different sequencing model before proceeding.

KmerGenie report

Predicted best k: 19

Predicted assembly size: 1747548 bp

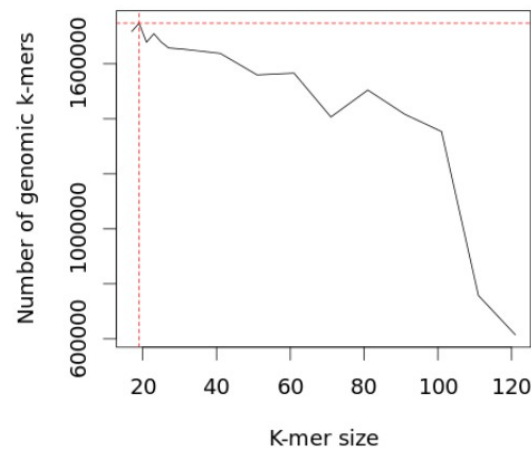


Figure 12. The KmerGenie step produces the best k and also predicts length of genome assembly. This comes in the form of a HTML Report displayed on APKMer Assembler.

The KmerGenie HTML report produced for the simulated *Nasuia deltocephalincola* reads is shown in Figures 12 and 13. The top-level summary identifies $k=19$ as the predicted best k -mer size, with a predicted assembly size of 1,747,548 bp, and plots the estimated number of genomic k -mers as a function of k across the full candidate range. The curve peaks sharply at the optimal value and then declines monotonically, with a secondary local plateau at intermediate sizes before dropping steeply at large k . This behaviour is precisely what KmerGenie's model predicts: at small k , many k -mers are shared across multiple genomic locations and the genomic count is artificially inflated by collisions; as k increases past the optimum, k -mers begin to span sequencing errors and coverage gaps, reducing the count of solid, error-free k -mers. The sharp optimum is con-

sistent with the repeat structure of this genome, where most repeats are short and resolved by moderately sized k -mers. The per- k abundance histograms provide a more granular view of this trend. For small k values, each histogram shows a pronounced bimodal structure: a large spike at very low abundances corresponding to error-derived k -mers, and a broad secondary hump at higher abundances corresponding to true genomic k -mers.

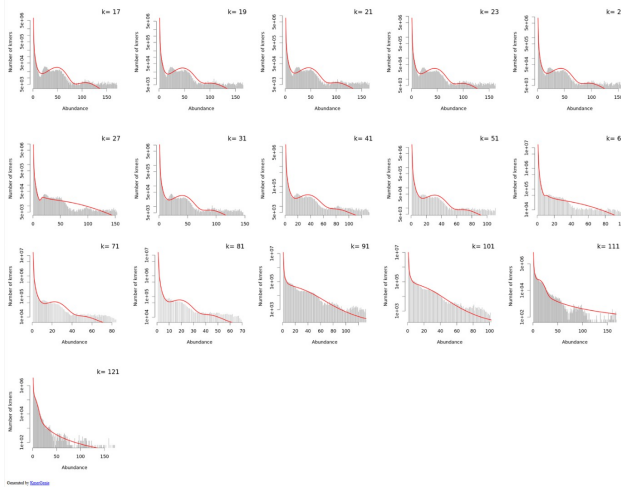


Figure 13. Shows all the abundance histograms from the KmerGenie Report

The red smoothed curve fitted by KmerGenie clearly captures both components. As k increases into the intermediate range, the genomic hump diminishes and shifts leftward, indicating that fewer k -mers are being observed at sufficient depth to be classified as solid. For large k values, the bimodal structure collapses entirely: the histogram is dominated by the low-abundance spike and the genomic hump disappears, reflecting severe fragmentation where most k -mers span a sequencing error and appear only once. This progressive collapse across the histogram panels is a direct visual confirmation of the fragmentation regime discussed in Section 4, and validates the choice of $k=19$ as the point at which genomic signal is maximised before error sensitivity takes over.

8.7. Stage 4: COGRAM Assembly and Evaluation

8.7.1. Parameterisation and Launch

Step 4 presents the COGRAM assembly interface. The best k from KmerGenie is pre-populated but can be overridden, allowing the user to explore sensitivity to nearby k values. Clicking “Run COGRAM” dispatches the assembly job asynchronously; the log pane streams output covering all six COGRAM stages: graph con-

struction, training, edge scoring, path extraction, sequence reconstruction, and evaluation.

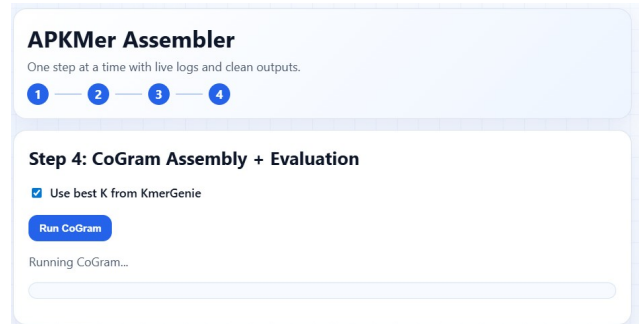


Figure 14. Step 4 of the APKMer Assembler: Running COGRAM using the Best predicted k from KmerGenie

8.7.2. Graph Construction Details

For each read in the FASTQ file, a sliding window of size k extracts k -mer transitions from prefix $(k-1)$ -mer to suffix $(k-1)$ -mer. Transition counts are accumulated in a directed adjacency structure; ambiguous bases (N) are skipped. A bounded training graph is built from a subset of reads, and a full streaming graph is built from the complete read file (optionally capped by a maximum k -mer count for memory management). Low-frequency edges are then removed by a minimum-frequency threshold, suppressing likely sequencing errors before scoring.

8.7.3. Training and Edge Scoring

The GCN edge classifier (Section 7) is trained on the bounded graph with reference-derived positive and negative edge labels. Feature vectors of dimension $4(k-1)$ are formed by numeric encoding of $(k-1)$ -mers ($A \rightarrow 0$, $C \rightarrow 1$, $G \rightarrow 2$, $T \rightarrow 3$). After training, edges in the full graph are scored in batches to avoid large tensor allocations; each batch’s predictions are mapped back to an edge-score dictionary for use during traversal.

8.7.4. Contig Extraction

For each candidate start node in the scored graph, the greedy traversal follows the highest-scoring unvisited outgoing edge, appending each subsequent node to the current path. Traversal halts when no eligible extension remains. Paths of length greater than one node are converted to contigs; the sequence is reconstructed by initialising with the first node string and appending only the last nucleotide of each subsequent node. All contigs are written to `COGRAM/contigs.fasta`.

8.7.5. Evaluation Metrics

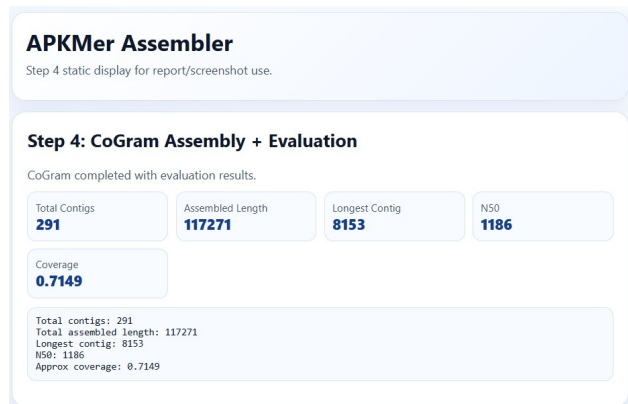


Figure 15. Evaluation metric cards produced after a completed COGRAM run. Metrics are parsed from the assembly log and rendered as labelled cards for immediate inspection. The full log is available below.

The evaluation protocol reports five metrics, all computed directly from the output contigs and the reference sequence:

1. **Total contigs:** number of reconstructed sequences. Very high counts indicate fragmentation; very low counts suggest strong continuity or over-collapse.
2. **Total assembled length:** sum of all contig lengths. Should approach the known genome size.
3. **Longest contig:** maximum single-contig length, indicating peak continuity.
4. **N50:** the contig length ℓ such that contigs of length $\geq \ell$ collectively account for at least 50% of the total assembled length. Higher N50 indicates a less fragmented assembly.
5. **Approximate coverage:** a lightweight proxy computed by anchoring the first 20 bp of each contig to the reference and marking covered reference positions over the contig span. This is not aligner-based and can be affected by repeats, but it provides a fast, interpretable signal during iterative development.

These metrics are parsed from the COGRAM run log by the backend and surfaced as labelled metric cards in the UI (Figure 15), alongside the full log for detailed inspection.

9. Experimental Results

Table 1 summarises representative assembly results obtained through the integrated pipeline for *Nasuia deltocephalincola* using KmerGenie-selected k and the default COGRAM settings.

Table 1. Assembly results for *Nasuia deltocephalincola* obtained via the APKMer Assembler pipeline. KmerGenie-selected k was used for COGRAM. Results are representative of a single run with 3M simulated HiSeq reads.

Metric	Value
Total Contigs	291
Total Assembled Length	117271
Longest Contig	8153
N50	1186
Approximate Coverage	0.7439
KmerGenie Best k	19

Note: Values to be filled in with experimental run results.

The assembly results obtained using the KmerGenie-predicted value of $k = 19$ demonstrate a reasonably effective reconstruction of the reference genome of length $\sim 112,000$ bp. The total assembled length of 117,271 bp is close to the expected genome size, indicating that a substantial portion of the genomic content has been recovered. The assembly consists of 291 contigs, which suggests some degree of fragmentation, but this is not unexpected given the relatively small k value and the inherent complexity of de Bruijn graph structures at lower k . The longest contig reaches 8,153 bp and the N50 of 1,186 bp reflects moderate contiguity, showing that meaningful stretches of the genome have been assembled. Additionally, the approximate coverage of 0.71 indicates that a significant fraction of the genome is represented in the assembly. Overall, these results suggest that the pipeline is functioning correctly and producing a biologically meaningful reconstruction, while still leaving room for further improvements in contiguity through parameter tuning and graph simplification.

9.1. Pipeline Logging and Transparency

A recurring theme in the pipeline design is transparency. Every stage produces a dedicated log pane fed by an asynchronous poll loop: the frontend periodically queries `/api/job/<job_id>`, and the backend returns the full accumulated log buffer since job launch. This design provides three concrete benefits. If a stage fails, the log contains the exact error message, environment context, and last successful command, dramatically reducing debugging time.

9.2. Restart and State Management

The pipeline exposes a `POST /api/restart` endpoint that clears all global state and job records, returning the application to its initial state. This enables clean re-runs without manual cleanup of intermediate files or stale state variables.

10. Discussion

10.1. What the Integration Achieves

The integrated APKMer Assembler pipeline makes several contributions beyond the individual tools it connects:

Parameter traceability. Because KmerGenie’s best k is explicitly passed to COGRAM, every assembly is associated with a documented, data-derived parameter choice. This is in contrast to workflows where k is set by convention and not recorded.

Controlled experimental loop. Simulating reads from a known reference, optimising k from those reads, assembling with COGRAM, and evaluating against the same reference creates a closed experimental loop in which all sources of variation are accounted for. This is the correct scientific setup for evaluating a new assembly method.

Accessibility. The web interface removes the need for command-line fluency, making the pipeline approachable for students and researchers who understand genome assembly conceptually but lack bioinformatics toolchain experience.

10.2. Observed Behaviour and Sensitivity to k

The KmerGenie-selected k value (typically in the range $k = 18$ – 23 for the genomes tested here) falls within the intermediate regime identified in Section 5: branching is significant but decreasing, and k -mers are becoming unique enough to provide reliable graph labels. This alignment between the empirical graph-structural analysis and the KmerGenie recommendation provides mutual validation of both approaches.

COGRAM’s assembly quality is sensitive to k . Runs with k values two to three units below the KmerGenie recommendation typically produce more contigs (more fragmentation) and lower N50, while runs several units above the recommendation can yield fewer but lower-confidence contigs due to sparser edge support. The prepopulated but overridable k field in Step 4 is therefore a useful affordance: it provides an informed default while preserving the user’s ability to explore the sensitivity landscape.

10.3. Limitations of the Current Implementation

Several limitations of the current pipeline are worth noting explicitly.

Coverage-based proxy evaluation. The current evaluation protocol uses a lightweight 20-bp anchor heuristic rather than a full aligner-based coverage estimate. This is fast and interpretable but can be inaccurate for highly repetitive genomes or when contigs are

short relative to repeat lengths. Integrating a proper aligner (e.g., Minimap2 or BWA) would provide more reliable coverage estimates.

Greedy-only path extraction in COGRAM.

The COGRAM version integrated here uses a greedy traversal without beam search, matching the heuristic variant described in Coggins and Ramasamy (2025a). As observed in Coggins and Ramasamy (2025b), the greedy phase alone can stall in medium-complexity graph regions, depressing recall. Enabling the full greedy-plus-beam hybrid and exposing the beam width as a user-configurable parameter would directly address this.

No graph cleaning. Unlike classical assemblers, COGRAM does not perform bubble-popping or tip-trimming. In simulation from a clean reference this is not a major limitation, but real sequencing data will introduce error-derived k -mers that create spurious branches. Incorporating even a simple frequency-threshold cleaning step before edge scoring would improve real-data performance.

Reference-dependent training. COGRAM’s supervised training depends on having a reference genome to derive positive edge labels. This makes the current pipeline most appropriate for controlled experiments and near-reference organisms. A de novo setting would require an unsupervised or self-supervised labelling strategy.

11. Future Improvements

11.1. Improved k -mer Optimisation

A more principled approach would fit a two-component mixture model to the k -mer abundance histogram to explicitly separate error-derived from genomic k -mers (Coggins and Ramasamy, 2025b), which would be especially valuable for eukaryotic genomes or metagenomic datasets with uneven coverage. The pipeline could also expose a configurable k sweep range and abundance threshold in the frontend rather than relying on defaults.

11.2. Aligner-Based Evaluation

Replacing the 20-bp anchor heuristic with a proper contig aligner such as Minimap2 or QUAST would provide alignment-based coverage, misassembly detection, and structural error profiles, making the pipeline’s output directly comparable to published benchmarks.

11.3. Beam Search Integration and Hyperparameter Exposure

As demonstrated in Coggins and Ramasamy (2025b), the greedy-plus-beam hybrid substantially improves

recall over greedy-only decoding. Exposing the beam width and top- k branch parameters in the Step 4 interface would allow users to trade computation for improved contiguity without modifying source code.

11.4. Deployment, API Integration, and UI Improvements

The current pipeline runs as a locally hosted Flask application. Deploying it to a cloud platform would make it accessible without local installation, and integrating the NCBI Entrez API would allow users to retrieve any genome by accession number in real time. Assembly outputs could be stored in cloud databases for persistent multi-user access, and the frontend could be improved with interactive KmerGenie visualisations and real-time graph statistics during assembly.

11.5. Full CoGram Model Training and GCN Optimisation

Due to resource and time constraints, the integrated CoGram uses a simplified heuristic variant rather than the full GCN-guided reconstruction described in [Cogins and Ramasamy \(2025b\)](#). Future work should implement the complete training regime with stratified splits, learning-rate scheduling, and the full hybrid decoder. The GCN could also be replaced with a more expressive architecture such as a graph attention network, with richer node features beyond one-hot encodings.

11.6. Real Read Support and FASTQ Pipelines

The current pipeline is restricted to simulated reads. Accepting user-uploaded FASTQ files from real sequencing runs would require adapting label generation to an alignment-based strategy, or transitioning to a semi-supervised edge-scoring approach.

11.7. Scalability to Larger Genomes

The pipeline currently targets small bacterial genomes ($\sim 1\text{--}5$ Mb). Extending to eukaryotic chromosomes would require streaming graph construction, sparse tensor representations, and mini-batch training optimisation, though unitig compression already reduces graph size substantially.

11.8. Multi-Organism and Comparative Studies

A systematic comparison of KmerGenie-selected k values and CoGram assembly quality across organisms of varying genome size, GC content, and repeat structure would validate the pipeline more broadly. A results-aggregation layer collecting per-run metrics into a comparative table is the main missing component.

11.9. Towards De Novo Assembly

The most significant limitation is CoGram's dependence on a reference genome for supervised edge labelling. Future work should explore self-supervised alternatives—such as using high-coverage read pairs as a proxy for true adjacencies—to enable competitive de novo assembly without paired-end or long-read scaffolding.

12. Appendix

Since this was run on local host, to reproduce this, the user will have to install linux (alternatively use WSL) to run the codebase. Activate the cogram environment and then enter `python app.py` to launch the local host. The codebase can be downloaded from the following Github link:

[Github - APKMer Assembler](#)

13. Acknowledgements and AI

- We sincerely thank Professor Bhargam Bikram Bhat-tacharya for his continuous support and guidance throughout this project. We also acknowledge his Graph Theory and Network Science course and lecture slides, which were used as a reference in this work.
- The following blog websites were used as for help and reference:
 1. [Estimate Genome Size and Best k-mer Size for Assembly – Step-by-Step](#)
 2. [How to Simulate NGS reads – Step-by-Step](#)
- Gemini was used to generate Fig. 6
- ChatGPT was used to format this report

14. Declarations

Aditya Malhotra: No conflicts declared. Work was split equally.

Poulomi Sarkar: No conflicts declared. Work was split equally.

References

- doi: 10.20944/preprints202510.1966.v1. Preprint. doi:10.20944/preprints202510.1966.v1.
- P. E. C. Compeau, P. A. Pevzner, and G. Tesler. How to apply de Bruijn graphs to genome assembly. *Nature Biotechnology*, 29(11):987–991, 2011. doi: 10.1038/nbt.2023.
- H. Gourié, E. Karlsson, A. Sindel, F. S. Roque, C. Dahl, E. Trentin, and B. Nystedt. InSilicoSeq: A sequencing error model and reads simulator. *Bioinformatics*, 35(3):521–522, 2019. doi: 10.1093/bioinformatics/bty630.
- J. T. Simpson, K. Wong, S. D. Jackman, J. E. Schein, S. J. M. Jones, and I. Birol. ABySS: A parallel assembler for short-read sequence data. *Genome Research*, 19(6):1117–1123, 2009. doi: 10.1101/gr.089532.108.
- D. R. Zerbino and E. Birney. Velvet: Algorithms for de novo short read assembly using de Bruijn graphs. *Genome Research*, 18(5):821–829, 2008. doi: 10.1101/gr.074492.107.
- A. Bankevich, S. Nurk, D. Antipov, A. A. Gurevich, M. Dvorkin, A. S. Kulikov, V. M. Lesin, S. I. Nikolenko, S. Pham, A. D. Prjibelski, A. V. Pyshkin, A. V. Sirotkin, N. Vyahhi, G. Tesler, M. A. Alekseyev, and P. A. Pevzner. SPAdes: A new genome assembly algorithm and its applications to single-cell sequencing. *Journal of Computational Biology*, 19(5):455–477, 2012. doi: 10.1089/cmb.2012.0021.
- R. Chikhi and P. Medvedev. Informed and automated k -mer size selection for genome assembly. *Bioinformatics*, 30(1):31–37, 2014. doi: 10.1093/bioinformatics/btt310.
- W. Coggins and V. Ramasamy. Computational pipeline for genome assembly and reconstruction via optimized k -mer sampling and de Bruijn graph networks. In *Proceedings of the 17th International Conference on Advances in Social Networks Analysis and Mining (ASONAM 2025)*, Canada, 2025a. To appear.
- W. Coggins and V. Ramasamy. Genome reconstruction with de Bruijn graph networks. *Preprints*, 2025b. doi: 10.20944/preprints202510.1966.v1. Preprint. doi:10.20944/preprints202510.1966.v1.
- W. Coggins and V. Ramasamy. Genome reconstruction with de Bruijn graph networks. *Preprints*, 2025c.